VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering

CBS3011 — Usability Design of Software Applications

EXPLICITE AI

AI-Powered Explainable Code Review and Security Learning Platform

Final Project Report — Group 6

Submitted by:

Dhruv Sharma (24BBS0063)

Rijul Kothawade (24BBS0230)

Submitted to: Swarnalatha P.

Academic Year: 2026–2027

Acknowledgements

We would like to express our sincere gratitude to our project supervisor, Swarnalatha P., for her guidance, support, and constructive feedback throughout the development of this project. Her insights into usability design and software engineering practice have significantly shaped the direction and quality of this work.

We are grateful to the School of Computer Science and Engineering at Vellore Institute of Technology for providing the academic environment and resources that made this project possible.

We acknowledge the open-source communities behind the technologies used in this project, particularly the Semgrep team for their static analysis engine and rule registry, the FastAPI and SQLAlchemy communities for their excellent documentation, and the Ollama project for making local large language model inference accessible.

Finally, we thank our families and fellow students for their encouragement and support throughout this project.

Dhruv Sharma and Rijul Kothawade

Vellore Institute of Technology, November 2026

Abstract

Explicite AI is an AI-powered, explainable code review and security learning platform designed for software developers and computer science students. The platform integrates Semgrep 1.181.0 static analysis with a FastAPI backend and a React 19 frontend to provide automated vulnerability detection, CVSS 3.1 severity scoring, AI-generated explanations, structured suggested fixes, and a personalised learning pathway.

The problem being addressed is the gap between automated code analysis tools and developer understanding: existing SAST tools identify vulnerabilities but rarely explain them in accessible terms or guide developers towards remediation. Explicite AI bridges this gap through five interconnected capabilities: (1) ZIP archive and Git repository static analysis; (2) CVSS 3.1-based vulnerability severity scoring; (3) five-field AI explanations adjustable to beginner, intermediate, or advanced levels; (4) AI-generated suggested fixes with explicit developer approval workflow and stale-source detection; and (5) a personalised learning system with an interactive AI tutor.

The platform is built on Python 3.14.3, FastAPI, PostgreSQL 16, React 19, TypeScript 5.7, Vite 8, and Tailwind CSS v4. AI capabilities are provided by a locally hosted Ollama inference server (llama3 model) with optional Anthropic API support. The backend comprises 20 test files with 426 tests passing and 1 skipped. The user interface implements the Explicite Midnight design system across 11 screens, following ISO 9241 usability principles and Nielsen's heuristics.

The project contributes to UN Sustainable Development Goals SDG 4 (Quality Education) through its educational AI tutor and personalised learning resources, SDG 9 (Industry and Innovation) through AI-augmented development tooling, and SDG 16 (Peace and Institutions) through its contribution to secure software development practices.

Table of Contents

Note: Update this table of contents in Microsoft Word by right-clicking and selecting 'Update Field' → 'Update entire table' before submission.

Abstract ii
List of Figures iii
List of Tables iv
1. Introduction 10
2. Problem Statement, Objectives and Scope 11
3. Literature Review 12
4. Existing Systems and Competitor Analysis 15
5. Proposed System and Architecture 17
6. Technology Stack 20
7. System and Database Design 22
8. Implementation and Functional Modules 25
9. User Interface and Usability Design 29
10. Testing and Results 34
11. Graphs and Analytical Visualisations 36
12. Sustainable Development Goals 40
13. Challenges, Limitations and Future Enhancements 41
14. Conclusion 43
References 44
Appendix A: Key API Endpoints 42
Appendix B: Team Contribution Statement 43

List of Figures

Figure 1: Login Screen 31
Figure 2: Main Dashboard with Project Metrics 31
Figure 3: Projects Screen 32
Figure 4: Code Analysis Screen 28
Figure 5: Findings Screen with Vulnerability Severity 28
Figure 6: AI Assistant Conversation 32
Figure 7: Learning Resources Screen 32
Figure 8: Application Settings Screen 33
Figure 9: Chart — Module Distribution 36
Figure 10: Chart — Test Suite Results 37
Figure 11: Chart — Feature Implementation Status 37
Figure 12: Chart — Vulnerability Severity Distribution 38
Figure 13: Chart — Competitor Feature Comparison 38
Figure 14: Chart — Technology Stack Files per Layer 39
Figure 15: Chart — SDG Contribution Feature Mapping 39

List of Abbreviations

Abbreviation	Expansion
AI	Artificial Intelligence
API	Application Programming Interface
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DB	Database
ER	Entity-Relationship
FastAPI	Fast Application Programming Interface (Python web framework)
FIRST	Forum of Incident Response and Security Teams
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JSONB	JSON Binary (PostgreSQL storage format)
JWT	JSON Web Token
LLM	Large Language Model
NVD	National Vulnerability Database
ORM	Object-Relational Mapper
OWASP	Open Web Application Security Project
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
SAST	Static Application Security Testing
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm
SQL	Structured Query Language
SSRF	Server-Side Request Forgery
UI	User Interface
UX	User Experience

UUID	Universally Unique Identifier
VIT	Vellore Institute of Technology
XAI	Explainable Artificial Intelligence

List of Tables

Table 1: System Requirements Summary..... 17
Table 2: Literature Review Summary..... 14
Table 3: Competitor Feature Comparison 16
Table 4: Technology Stack Summary..... 20
Table 5: Database Entity Overview..... 23
Table 6: API Endpoint Reference..... 45
Table 7: Test Suite Summary 35
Table 8: SDG Feature Mapping..... 40

Chapter 1: Introduction

1.1 Background

Software vulnerabilities continue to be a primary cause of security incidents worldwide. Static Application Security Testing (SAST) tools automate the detection of vulnerability patterns in source code, enabling developers to identify potential issues before deployment. However, existing SAST tools present results as rule identifiers and severity ratings with minimal explanation, creating a gap between detection and developer understanding.

Junior developers and students often lack the security background needed to interpret SAST output effectively. A finding labelled 'CWE-89: SQL Injection' communicates little to a developer unfamiliar with injection attack classes, and existing tools provide no structured guidance on remediation or on building the underlying security knowledge. This represents a significant usability problem with real security consequences.

1.2 Explicite AI

Explicite AI is a web application that integrates SAST with AI-powered vulnerability explanations, structured suggested fixes, and an interactive learning system. The platform processes source code submitted as a ZIP archive or public Git repository, runs Semgrep 1.181.0 static analysis, scores findings using CVSS 3.1, and presents results with AI-generated explanations tailored to the developer's stated expertise level.

The platform is named 'Explicite AI' to reflect its core design principle: making security analysis explicit and comprehensible, rather than opaque and technical. The backend is built with Python 3.14.3 and FastAPI; the frontend uses React 19, TypeScript 5.7, Vite 8, and Tailwind CSS v4. PostgreSQL 16 serves as the database. AI capabilities use a locally hosted Ollama inference server as the default provider, with Anthropic API support as an alternative.

1.3 Report Structure

This report is structured as follows: Chapters 2 and 3 cover objectives, scope, and literature review. Chapters 4 and 5 present competitor analysis and the proposed system architecture. Chapters 6 and 7 detail the technology stack and database design. Chapter 8 covers functional implementation; Chapter 9 addresses UI and usability design. Chapter 10 presents testing results. Chapter 11 provides analytical visualisations. Chapters 12 and 13 discuss SDG contributions and project limitations. Chapter 14 concludes the report.

Chapter 2: Problem Statement, Objectives and Scope

2.1 Problem Statement

Automated code analysis tools detect vulnerabilities efficiently but communicate findings in technical language that assumes security expertise. Developers, particularly students and early-career engineers, face three related problems: (1) they cannot interpret SAST findings without additional research; (2) they do not receive guidance on remediation specific to their codebase; and (3) there is no systematic mechanism for building the underlying security knowledge to avoid similar vulnerabilities in the future.

2.2 Primary Objectives

- Develop an automated static analysis pipeline supporting ZIP archive and Git repository source submission.
- Implement CVSS 3.1 severity scoring for detected findings.
- Integrate an AI service providing structured, adjustable-level vulnerability explanations.
- Implement an AI-generated suggested fix workflow with mandatory developer approval.
- Build a personalised learning system with recommendations, learning paths, and AI tutor.
- Design a consistent, heuristics-guided user interface across all platform screens.
- Provide project management and role-based access control for team collaboration.
- Implement SSRF protection for Git repository analysis and ZIP bomb detection for archive analysis.
- Achieve comprehensive backend test coverage with no failures.

2.3 Project Scope

The platform covers the following within scope:

- Source code analysis for Python, Java, JavaScript, and TypeScript files.
- Public HTTPS Git repository cloning with SSRF and size protection.
- CVSS 3.1 base score computation from Semgrep rule metadata.
- AI explanations and fixes using Ollama (llama3) or Anthropic (claude-haiku-4-5).
- Learning resources, paths, tutor sessions, and progress tracking.
- JWT HS256 authentication and project membership with viewer/developer/owner roles.
- Local development deployment targeting a single-user or small-team environment.

The following are explicitly out of scope:

- DAST (dynamic application security testing) or runtime monitoring.
- CI/CD pipeline integration or automated analysis triggers.
- Private repository authentication (OAuth with GitHub/GitLab).
- Formal cloud deployment, horizontal scaling, or multi-tenancy.
- Code coverage measurement (infrastructure dependency not available).
- CVSS 4.0 scoring (vectors are stored but not scored with the v4.0 formula).

Chapter 3: Literature Review

3.1 Overview

This literature review examines eight publications relevant to AI-powered code review, automated vulnerability detection, and AI-assisted developer learning. Four papers were supplied with the project; four additional papers were identified through academic database searches and verified against their published DOIs and venues.

3.2 AI-Powered Code Review

Rasheed et al. (2024) [1] present a multi-agent GPT-4-based code review system evaluated on real developer codebases. Their system provides structured review comments across multiple quality dimensions. Results show measurable improvement in both code quality metrics and developer learning outcomes, supporting the Explicite AI hypothesis that AI explanation improves developer understanding alongside vulnerability detection.

İçöz and Biricik (2025) [2] propose a hybrid system combining a GraphCodeBERT LLM with symbolic reasoning components for automated code review, achieving 68.7% accuracy on their evaluation dataset. Their survey of 287 developers finds that 91% believe AI tools will improve software quality. The work supports the use of LLMs for code quality assessment while noting that model accuracy remains a limiting factor.

Vijayvergiya et al. (2024) [3] describe AutoCommenter, an AI-assisted code review system deployed internally at Google to over 10,000 developers. Their work demonstrates that AI review assistance at industrial scale is feasible, though they note challenges in calibrating review tone and specificity for different development contexts. Limbani et al. (2025) [4] demonstrate a P@1 score of 95% and a 33% reduction in review cycle time in a controlled study, providing quantitative evidence for AI review tools.

3.3 Vulnerability Detection and SAST

Li et al. (2018) [5] introduce VulDeePecker, a deep learning system for binary code vulnerability detection using code gadgets as input features. The system identifies four previously unreported CVEs in real software, establishing that deep learning models can detect subtle vulnerability patterns that escape traditional rule-based tools. This work informs Explicite AI's use of Semgrep rule-based detection complemented by AI explanation.

Bennett et al. (2024) [6] evaluate commercial and open-source SAST tools including Semgrep on a curated vulnerability dataset. Their key finding is that individual tools detect only 11–26% of vulnerabilities independently, while their Semgrep* combination approach reaches 44.7% detection. This directly informs Explicite AI's design: Semgrep is used as the analysis engine with the understanding that no single tool achieves complete detection coverage.

3.4 AI-Assisted Fix Generation

Fan et al. (2023) [7] evaluate Codex and other LLMs on automated program repair benchmarks. Their results show that LLMs achieve comparable repair rates to specialised automated repair tools on certain benchmark suites but generate plausible-looking but incorrect fixes in a significant proportion of cases. This finding directly motivates Explicite AI's design decision to present fixes as suggestions requiring explicit developer approval, rather than applying them automatically.

Mao et al. (2024) [8] present LLMVulExp, a framework for explainable vulnerability detection. Using a custom evaluation rubric with three criteria (Accuracy, Clarity, Actionability), they demonstrate that their approach achieves over 90% F1 on vulnerability identification with substantially improved explanation quality compared to baseline LLMs. The five-field explanation structure in Explicite AI (what, why, how_it_arises, remediation, caveats) is directly inspired by this work's evaluation framework.

3.5 Research Gap and Contribution

The literature establishes that: (a) AI code review is effective at improving quality but requires careful UX design; (b) SAST tools have limited individual detection coverage; (c) AI-generated fixes require human validation; and (d) structured AI explanations measurably improve developer comprehension. A gap exists in integrated platforms that combine all four capabilities in a usability-centred design with an embedded learning pathway. Explicite AI addresses this gap.

Ref	Authors	Year	Venue	Key Contribution
[1]	Rasheed et al.	2024	arXiv	Multi-agent GPT-4 code review; dual quality + learning benefit
[2]	İçöz & Biricik	2025	IEEE	GraphCodeBERT hybrid; 68.7% accuracy; 91% developer survey
[3]	Vijayvergiya et al.	2024	ACM AIware	AutoCommenter: 10,000+ Google developers; industrial scale
[4]	Limbani et al.	2025	IJCRT	P@1=95%; 33% review time reduction in controlled study
[5]	Li et al.	2018	NDSS	VulDeePecker: deep learning finds 4 unreported CVEs
[6]	Bennett et al.	2024	ACM EASE	Semgrep*: tools detect 11–26% individually; combo: 44.7%
[7]	Fan et al.	2023	ICSE	LLMs for repair: comparable to specialised tools; approval needed

[8]	Mao et al.	2024	arXiv	LLMVulExp: >90% F1; Accuracy/Clarity/Actionability metrics

Table 2: Literature Review Summary

Chapter 4: Existing Systems and Competitor Analysis

4.1 Overview of Competing Tools

The static analysis and AI code review market includes both commercial tools (SonarQube, Snyk Code, Checkmarx) and open-source tools (Semgrep Community Edition, GitHub CodeQL). Each tool addresses different aspects of the vulnerability management workflow. This analysis focuses on four tools most relevant to Explicite AI's use case: developer-facing static analysis with AI augmentation.

SonarQube: Enterprise SAST platform with multi-language support, CVSS scoring on Commercial/Enterprise tiers, dashboards, and CI/CD integration. Widely used in professional teams. AI-generated explanations and fix suggestions are not available in the Community Edition.

Snyk Code: Developer-first SAST with fix suggestions integrated into IDEs and CI/CD pipelines. Offers partial AI assistance for certain fix patterns. Strong language support. Primarily commercial with limited free tier.

GitHub CodeQL: Dataflow-based query language enabling deep semantic analysis. Used in GitHub Advanced Security. Excellent CI/CD integration. No AI explanations or learning pathway. Requires CodeQL query authorship for custom rules.

Semgrep CE: Open-source SAST with pattern-based rule engine and a large public registry. Supports Python, Java, JS, TS, and many other languages. No AI explanations or suggested fixes in the open-source version. Forms the analysis engine inside Explicite AI.

4.2 Feature Comparison

Feature	Explicite AI	SonarQube	Snyk Code	CodeQL	Semgrep CE
Static Analysis	✓	✓	✓	✓	✓
Vulnerability Detection	✓	✓	✓	✓	✓
CVSS Scoring	✓	~ (paid)	~ (paid)	~ (paid)	✗
AI Explanations	✓	✗	~ (paid)	✗	✗
Suggested Fixes	✓ (advisory)	✗	✓ (paid)	✗	✗
Learning Pathway	✓	✗	✗	✗	✗
AI Tutor	✓	✗	✗	✗	✗
CI/CD Integration	✗	✓	✓	✓	✓
Free to Deploy	✓	~ (CE only)	✗	~ (limited)	✓

Table 3: Competitor Feature Comparison (✓ = supported, ~ = partial/paid tier, ✗ = not available)

Source: Official product documentation, 2025–2026

4.3 Competitive Differentiation

Explicite AI's primary differentiator is the combination of AI explanations, advisory fix generation, and an integrated learning pathway in a single free-to-deploy platform. No existing open-source tool offers AI-generated explanations or a structured learning pathway. Commercial alternatives (SonarQube Enterprise, Snyk Code) offer some AI features but without educational integration and at significant licence cost.

Explicite AI's key limitation relative to commercial tools is the absence of CI/CD pipeline integration, which prevents it from fitting naturally into automated DevOps workflows. This is identified as a future enhancement.

Chapter 5: Proposed System and Architecture

5.1 System Overview

Explicite AI is a three-tier web application: a React single-page frontend, a FastAPI REST backend, and a PostgreSQL 16 database. The backend is augmented by two external integrations: Semgrep (static analysis engine, invoked as a managed subprocess) and an AI provider (Ollama local inference server or Anthropic API). The system processes developer-submitted source code through a multi-stage analysis pipeline and persists results, explanations, and fixes in the database.

5.2 Component Architecture

Component	Technology	Responsibility
Frontend SPA	React 19 + TypeScript + Vite 8	11-screen UI, navigation via NavCtx, API calls via api.ts
API Gateway	FastAPI + Python 3.14.3	Auth, routing, validation, background job dispatch
Analysis Engine	Semgrep 1.181.0 (subprocess)	SAST scanning of extracted source files
AI Service	Ollama (llama3) / Anthropic	Explanations, fix generation, chat, tutoring
Database	PostgreSQL 16 (port 5434)	Users, projects, jobs, findings, fixes, learning data
ORM/Migrations	SQLAlchemy 2.x async + Alembic	Non-blocking DB access, 9-migration schema chain
Git Service	git clone + Python validation	HTTPS-only repo cloning with SSRF protection
Archive Service	Python zipfile + validation	ZIP validation, decompression bomb detection

Table 1: System Component Architecture

5.3 Analysis Pipeline

The code analysis pipeline consists of eight sequential stages:

1. Source Submission: Developer uploads a ZIP archive (max 10 MB) or submits an HTTPS Git repository URL.

2. Input Validation: Archive: ZIP magic bytes, compression ratio (max 100:1), entry count (max 1,000), size (max 50 MB uncompressed). Git: URL scheme, credentials, branch allowlist regex, DNS resolution, private IP blocking.

3. Source Extraction: Archive extraction to isolated temporary directory. Git: git clone --depth 1 with GIT_TERMINAL_PROMPT=0, .git/ removed post-clone.

4. Job Creation: AnalysisJob record created with status 'queued'. HTTP 202 returned immediately.

5. Semgrep Execution: Semgrep 1.181.0 invoked as subprocess (shell=False) with fixed argument list. JSON output parsed into Finding records.

6. Metric Computation: Technical debt calculated (Vulnerability=30min, Security Hotspot=20min, Bug=15min, Code Smell=5min). Duplication detection via hash-based block comparison. Risk score computed.

7. CVSS Scoring: CVSS 3.1 vectors from Semgrep rule metadata scored per FIRST specification. CVSS 4.0 vectors stored without scoring.

8. Result Persistence: Job status set to 'completed'. Findings, metrics, and risk scores persisted in PostgreSQL.

5.4 Security Architecture

Security is implemented in multiple layers. Authentication uses JWT HS256 bearer tokens stored in sessionStorage. All API routes except /health and /auth/login require a valid token. Project-scoped routes additionally verify membership role. AI output is validated for required fields before storage; malformed responses return HTTP 502. Source file paths are sanitised from failure messages before storage. Semgrep results are treated as untrusted data. No AI-generated code is executed.

5.5 System Requirements

Requirement	Detail
Backend runtime	Python 3.14.3, FastAPI, uvicorn
Database	PostgreSQL 16 (Docker, port 5434)
Analysis engine	Semgrep 1.181.0 (installed in backend venv)
AI provider	Ollama with llama3 model at localhost:11434
Frontend build	Node.js 20+, Vite 8, npm
Frontend runtime	Modern browser (Chrome, Firefox, Edge)
Source file size	Max 10 MB ZIP upload; max 100 MB cloned repository

Table 1: System Requirements Summary

5.6 System Architecture Diagram

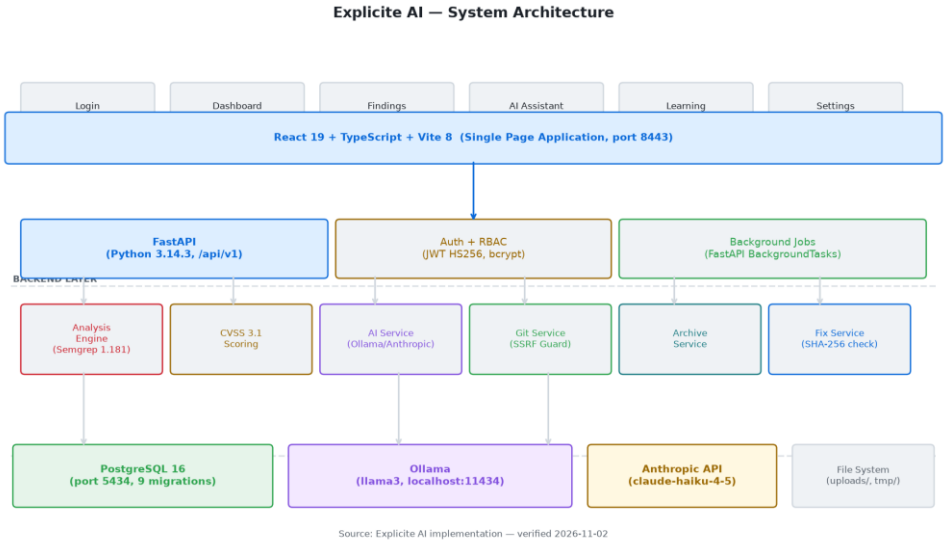


Figure — System Architecture: Frontend SPA → FastAPI Backend → PostgreSQL + Semgrep + AI Providers

(Source: Implementation architecture, verified 2026-11-02)

5.7 Analysis Workflow Diagram

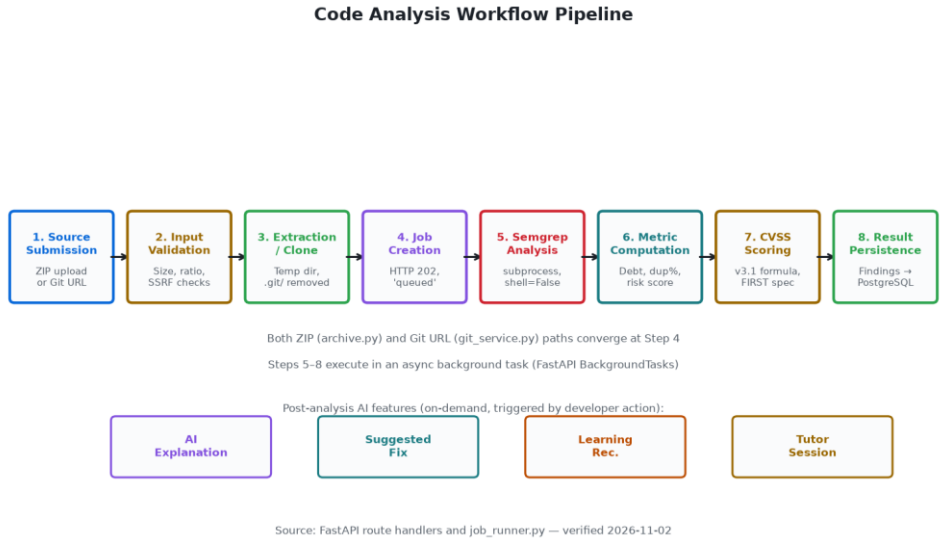


Figure — Code Analysis Workflow: 8-stage pipeline from source submission to result persistence

(Source: job_runner.py and analysis.py, verified 2026-11-02)

Chapter 6: Technology Stack

6.1 Technology Selection

The Explicite AI technology stack was selected to satisfy three constraints: (1) non-blocking I/O throughout the API layer (required by long-running analysis jobs); (2) type safety and developer ergonomics across both frontend and backend; and (3) minimal operational complexity for local development deployment.

Layer	Technology	Version	Role
Frontend	React	19	Component UI, concurrent rendering
Frontend	TypeScript	5.7	Static type checking
Frontend	Vite	8.0.5	Build tool, HMR dev server (port 8443)
Frontend	Tailwind CSS	v4	Utility styling + Explicite Midnight design tokens
Frontend	Inter + JetBrains Mono	—	UI text + code display typography
Backend	Python	3.14.3	Primary backend language
Backend	FastAPI	≥0.115	Async REST API framework
Backend	Pydantic	v2	Request/response validation, settings
Database	PostgreSQL	16	Relational data store (Docker, port 5434)
Database	SQLAlchemy	2.x async	Async ORM with AsyncSession
Database	asyncpg	≥0.30	PostgreSQL async driver
Database	Alembic	≥1.14	Schema migrations (9 total)
Analysis	Semgrep	1.181.0	SAST engine (subprocess, shell=False)
AI	Ollama	—	Local LLM inference server
AI	llama3	8B	Default language model
AI	Anthropic API	optional	claude-haiku-4-5 (cloud alternative)
Auth	PyJWT	≥2.9	JWT HS256 token generation and validation
Auth	bcrypt	≥4.2	Password hashing
Testing	pytest + asyncio	≥8.3	Backend test suite
Testing	httpx	≥0.28	Async HTTP test client

Table 4: Technology Stack Summary (Verified from requirements.txt and package.json)

6.2 AI Provider Architecture

The AI layer supports three selectable providers controlled by the LLM_PROVIDER environment variable: ollama (default in production .env), anthropic (requires

ANTHROPIC_API_KEY), and none (returns HTTP 503 with an honest 'not configured' message — the safe default in .env.example). All providers implement a common interface in ai_service.py. The fix generation endpoint passes LLM_FIX_MAX_TOKENS=2048 instead of LLM_MAX_TOKENS=512, accommodating longer code output without affecting other AI endpoints.

Chapter 7: System and Database Design

7.1 Database Schema

The database schema is managed through nine Alembic migrations from initial schema (18ab260079b1) to the current head (c1d2e3f4a5b6). The schema comprises six core tables and three auxiliary tables. All primary keys use UUID v4.

Table	Key Columns	Notes
users	id (UUID PK), email, password_hash, system_role, department, experience_level, is_email_verified, created_at	Preferences stored as jsonb field in newer migrations
projects	id, name, description, owner_id (FK→users), created_at	Members added via project_memberships join table
project_memberships	project_id (FK), user_id (FK), role (viewer/developer/owner)	Many-to-many join table
analysis_jobs	id, project_id (FK), status, engine_name, engine_version, bugs_count, vulnerabilities_count, code_smells_count, security_hotspots_count, technical_debt_minutes, duplications_pct, repo_url, source_path, source_branch, source_revision, failure_reason, timestamps	status: queued→running→completed|failed
findings	id, analysis_job_id (FK), project_id (FK), engine_severity, risk_level, risk_score, issue_type, title, rule_key, file_path, line_number, description, status, cwe_id, owasp_category, cvss_score, cvss_vector, cvss_version, ai_explanation, created_at, updated_at	status: Open|In Review|Resolved
suggested_fixes	id, finding_id (FK), proposed_lines (JSONB), rationale, caveats, original_hash, status (pending/approved/rejected)	Hash enables stale-source detection at approval time
learning_resources	id, title, url, description, resource_type, tags (JSONB), cwe_ids, rule_keys, issue_types	Links to findings by rule_key and issue_type
learning_paths	id, title, description, difficulty	Ordered sequences of resources
tutor_sessions / messages	session_id, user_id (FK), resource_id (FK), messages: role, content, created_at	Conversation history persisted per user per resource

Table 5: Database Entity Overview (Verified from SQLAlchemy model files)

7.1.1 Entity-Relationship Diagram

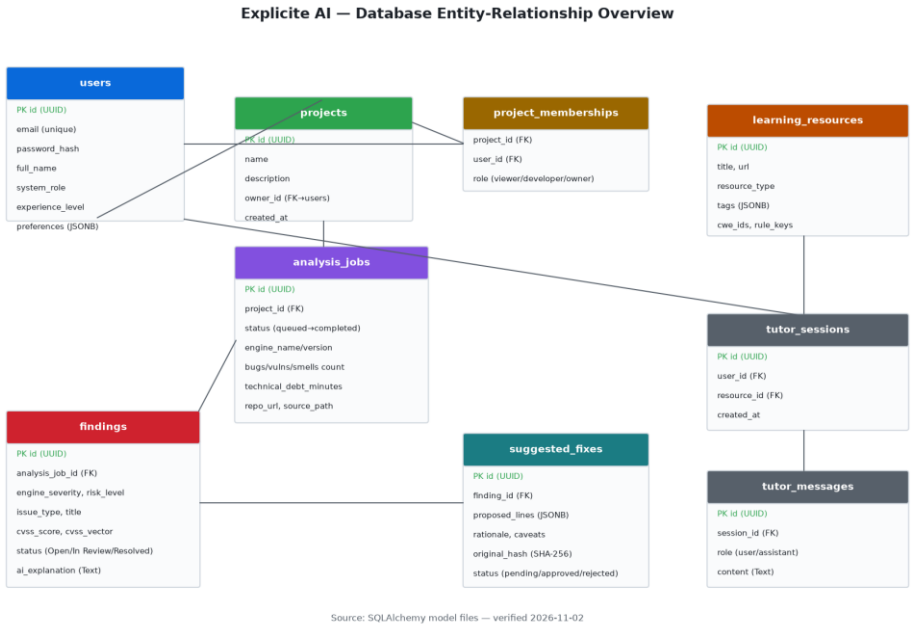


Figure — Database ER Diagram: Entities and their relationships

(Source: SQLAlchemy models, verified 2026-11-02)

7.2 Migration History

Nine Alembic migrations manage the schema evolution from the initial design to the current production schema. Migrations are authored as Python scripts with async support and are safe to run on a fresh database or apply incrementally.

`18ab260079b1` (initial_schema): Users, projects, memberships, analysis jobs, findings

`53ef0c95f7a6` (add_suggested_fixes_table): Adds suggested_fixes table

`a7f3b90d12e8` (add_tutor_sessions_and_messages): Adds tutor_sessions and tutor_messages

`4df931a7000b` (add_learning_path_resources_table): Adds learning_path_resources join table

`5d7e502760f4` (add_cvss_vector_and_version_to_findings): Adds cvss_vector, cvss_version to findings

`e8a4f1b9c253` (add_user_preferences): Adds preferences JSONB column to users

`caf27a49317a` (add_source_path_to_analysis_jobs): Adds source_path (internal, not exposed in API)

`c1d2e3f4a5b6` (add_repo_url_to_analysis_jobs): Adds repo_url for Git repository analysis — current HEAD

`(8 named above)` (total: 9 migration files): Additional migration in versions/ directory

7.3 Role-Based Access Control

Access control is enforced at two levels. System role (member/admin) is stored on the user record; system administrators bypass membership checks. Project role (viewer/developer/owner) is stored in project_memberships. The get_project_membership() dependency enforces the

required minimum role for each endpoint. Fix approval and Git/ZIP analysis submission require developer or owner role.

Chapter 8: Implementation and Functional Modules

8.1 Module Overview

The platform comprises 11 frontend screens, 8 API route modules, 13 service modules, and 6 database models.

8.2 Authentication

POST /api/v1/auth/login accepts JSON with email and password, validates against the bcrypt hash, and returns a signed JWT (HS256, expiry: JWT_ACCESS_TOKEN_EXPIRE_MINUTES). The token is stored in sessionStorage under the key explicite_token on the frontend. GET /api/v1/auth/me returns the authenticated user's profile. PATCH /api/v1/auth/me updates preferences including AI explanation level and theme.

8.3 Code Analysis

8.3.1 ZIP Archive Analysis

POST /api/v1/projects/{project_id}/analyse accepts a ZIP archive (max 10 MB). The archive service validates: ZIP magic bytes (PK\x03\x04), compression ratio (max 100:1 per entry to prevent decompression bombs), entry count (max 1,000), and total uncompressed size (max 50 MB). Valid archives are extracted to an isolated temporary directory. HTTP 202 is returned immediately; analysis runs as a FastAPI background task.

8.3.2 Git Repository Analysis

POST /api/v1/projects/{project_id}/analyse/git accepts a JSON body with repo_url and branch. Validation: HTTPS scheme only; embedded credentials rejected; branch validated against ^[a-zA-Z0-9][a-zA-Z0-9.\-/]*$; DNS resolution performed; all resolved IPs checked against private ranges (RFC 1918, loopback 127.0.0.0/8, link-local 169.254.0.0/16, and other non-routable ranges). Clone uses git clone --depth 1 with GIT_TERMINAL_PROMPT=0; .git/ directory removed post-clone.

8.4 Findings and CVSS Scoring

The findings list endpoint (GET /projects/{id}/issues) returns all findings from the latest completed analysis job. Findings support filtering by severity (Critical/High/Medium/Low — case-sensitive), issue type (Vulnerability/Bug/Code Smell/Security Hotspot), status (Open/In Review/Resolved), and free-text search on title and file path. Results are sorted by severity (Critical first) then title.

CVSS 3.1 base scores are computed in cvss_scoring.py following the FIRST specification. The Roundup function is taken verbatim from the official FIRST JavaScript reference implementation. CVSS 4.0 vectors are detected by prefix and stored without scoring. Findings without CVSS data (code quality findings) display no score.

8.5 AI Features

8.5.1 Vulnerability Explanations

POST /projects/{id}/issues/{iid}/explain calls the AI service with an explanation level parameter (beginner/intermediate/advanced). The response is a five-field structured object: what (the finding in plain terms), why (the security risk), how_it_arises (how this pattern occurs), remediation (concrete steps), and caveats (limitations, false-positive risk). Each field is limited to 600 characters. Invalid (missing-field) responses return HTTP 502.

8.5.2 Suggested Fix Workflow

Fix generation (POST .../fix) sends the finding context and surrounding source code to the AI with LLM_FIX_MAX_TOKENS=2048. The original source file's SHA-256 hash is stored with the fix. At approval time, the current source file's hash is recomputed; a mismatch returns HTTP 409 (stale source). Approved fixes are never applied automatically. GET .../download returns an in-memory ZIP containing the patched file for the developer to apply manually.

8.5.3 AI Assistant and Tutor

POST /projects/{id}/assistant provides a project-scoped AI chat. The learning tutor (POST /learning/tutor/{session_id}/chat) maintains per-user conversation history anchored to a specific learning resource, with the resource topic in the AI system prompt. Sessions and messages are persisted in the database, surviving browser sessions.

8.6 Learning Module

Learning resources are tagged by vulnerability category, CWE ID, and Semgrep rule key. The recommendation algorithm scores resources against the user's open findings: rule_key match scores 3 points, issue_type match 2 points, tag match 1 point. Completed resources are excluded. Resources are presented in order of highest score. Learning paths provide ordered resource sequences with per-user progress tracking.

8.7 Selected Code: CVSS 3.1 Scoring

The following excerpt from cvss_scoring.py illustrates the CVSS 3.1 base score computation, implementing the formula from the FIRST specification:

```
def score_cvss_vector(vector: str) -> CvssResult:
    version = _detect_version(vector)
    if version != '3.1':
        return CvssResult(score=None, version=version, severity=None,
error=None)
    parts = _parse_v31(vector)
    iss = 1 - (1-C(parts['C'])) * (1-C(parts['I'])) * (1-C(parts['A']))
    if parts['S'] == 'U':
        impact = 6.42 * iss
```

```
    else:
        impact = 7.52 * (iss - 0.029) - 3.25 * (iss - 0.02) ** 15
    exploitability = (
        8.22 * AV(parts['AV']) * AC(parts['AC'])
        * PR(parts['PR'], parts['S']) * UI(parts['UI']))
    if impact <= 0:
        return CvssResult(score=0.0, severity='None', version='3.1',
error=None)
    if parts['S'] == 'U':
        base = _roundup(min(impact + exploitability, 10))
    else:
        base = _roundup(min(1.08 * (impact + exploitability), 10))
    return CvssResult(score=base, severity=_v31_severity(base),
version='3.1', error=None)
```

8.8 Selected Code: Technical Debt Calculation

The technical debt estimate from job_runner.py uses configurable weights per issue type:

```
# Technical debt estimates — minutes per finding by issue type.
# Labeled 'estimate' in API responses; no industry-standard claim is made.
_DEBT_MINUTES_PER_TYPE: dict[str, int] = {
    'Vulnerability': 30,
    'Security Hotspot': 20,
    'Bug': 15,
    'Code Smell': 5,
}

def _calculate_technical_debt(findings: list) -> int:
    return sum(_DEBT_MINUTES_PER_TYPE.get(f.issue_type, 5) for f in
findings)
```

8.9 Selected Code: AI Provider Abstraction

The AI service provider is selected at runtime via LLM_PROVIDER environment variable:

```
class OllamaProvider:
    async def _call(self, system: str, user_message: str,
                    json_mode: bool = False,
                    max_tokens: int | None = None) -> str:
        body = {
            'model': self._model,
            'messages': [{'role': 'system', 'content': system},
```

```
                    {'role': 'user', 'content': user_message}],
        'stream': False,
        'options': {'num_predict': max_tokens or self._max_tokens},
    }
    # json_mode: format='json' in request body
    # Timeout: settings.LLM_TIMEOUT_SECONDS (default 30)
```

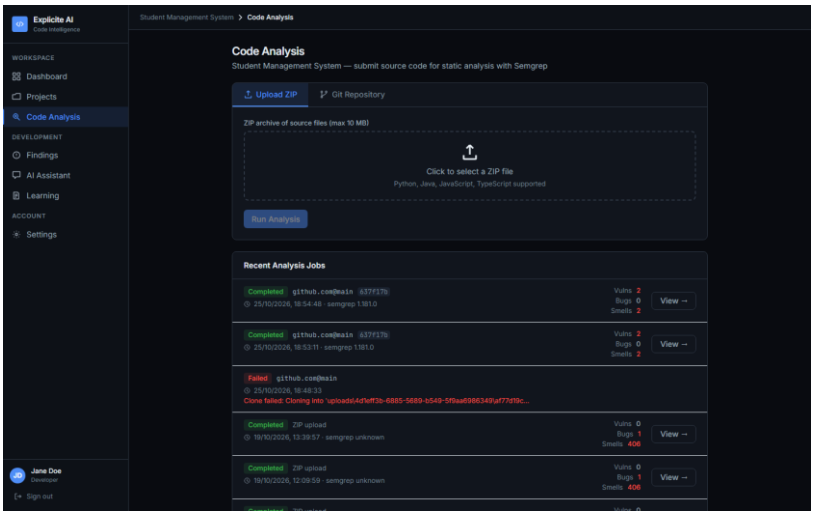


Figure 4: Code Analysis Screen — Showing analysis history for the selected project (Source: Live application screenshot, 2026-11-02)

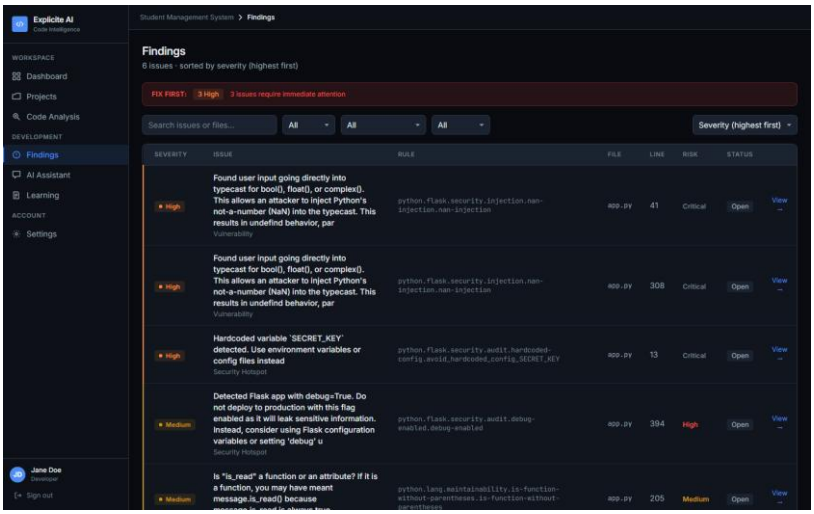


Figure 5: Findings Screen — Vulnerability findings with severity classifications (Source: Live application screenshot, 2026-11-02)

Chapter 9: User Interface and Usability Design

9.1 Design Principles

The user interface is designed in accordance with ISO 9241-110 usability principles and Nielsen's ten usability heuristics. The primary design goals are: clarity (surface what matters), discoverability (know where to go next), consistency (identical semantics across all screens), and feedback (always know what the system is doing).

9.2 Explicite Midnight Design System

All visual design is governed by the Explicite Midnight design system, implemented as CSS custom properties with the prefix --xm-. Dark mode is the default; light mode is provided by overriding the full token set under html[data-theme="light"]. The user can switch themes in the Settings screen; the preference is persisted via PATCH /api/v1/auth/me.

9.3 Navigation and Screens

Navigation is managed by the NavCtx React context using useState<Screen>. The sidebar provides persistent access to all 11 screens. The navigation labels as implemented are: Dashboard, Projects, Code Analysis, Findings, AI Assistant, Learning, Settings. (Note: the sidebar label for the findings list is 'Findings', not 'Issues' — verified from the running application.)

Screen	Sidebar Label	Primary Purpose
Login	—	JWT authentication
Projects	Projects	Project list, creation, member management
Dashboard	Dashboard	Aggregated metrics for selected project
Code Analysis	Code Analysis	ZIP/Git submission, job status, history
Findings	Findings	Paginated filterable findings list
Issue Detail	(from Findings)	Full finding: CVSS, AI explanation, fix controls
AI Assistant	AI Assistant	Project-scoped AI chat
Suggested Fix	(from Findings)	Diff view, caveats, approval/rejection
Learning	Learning	Resources, paths, recommendations, progress
Tutor Session	(from Learning)	Multi-turn AI chat per learning resource
Settings	Settings	Profile, AI level, theme toggle

9.4 Heuristics Application

Visibility of system status: Job status polling during analysis. Loading indicators during AI generation. Error states display actionable messages rather than HTTP status codes.

User control and freedom: Fix approval is always explicit. Fixes can be rejected. Navigation always allows return to any screen.

Consistency and standards: --xm-* tokens applied uniformly. Severity badges use identical colours across all 11 screens.

Error prevention: ZIP validation prevents dangerous archives. Git URL validation prevents SSRF. Form validation provides field-level error messages.

Recognition over recall: Sidebar labels are always visible. Severity uses colour and text labels consistently.

Flexibility and efficiency: AI explanation level (beginner/intermediate/advanced) adapts to expertise. Findings support multi-filter search.

9.5 Design Token Reference

The Explicite Midnight design token set (verified from index.css):

Token	Dark Value	Light Value	Semantic Use
--xm-bg-app	#090B10	#F6F8FA	Application background
--xm-bg-sidebar	#0D1117	#FFFFFF	Navigation sidebar background
--xm-bg-surface	#11151D	#FFFFFF	Card and panel background
--xm-bg-elevated	#1B2330	#F0F3F6	Elevated / modal surfaces
--xm-accent	#4F8EF7	#0969DA	Primary interactive elements
--xm-text	#E6EDF3	#1C2128	Primary body text
--xm-muted	#8B949E	#6E7781	Secondary / placeholder text
--xm-severity-critical	Red	Red	Critical severity badge
--xm-severity-high	Amber	Amber	High severity badge
--xm-severity-medium	Cyan	Cyan	Medium severity badge
--xm-severity-low	Green	Green	Low severity badge

9.6 Application Screenshots

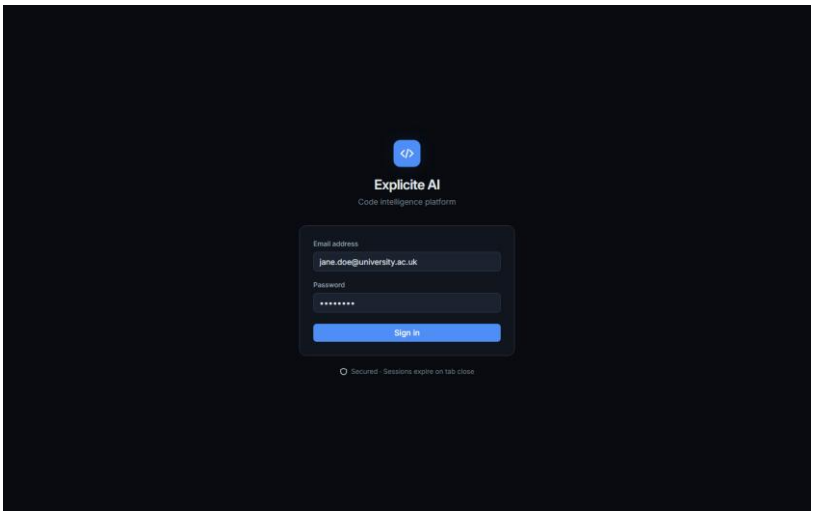


Figure 1: Login Screen — Email/password authentication with JWT storage in sessionStorage

(Source: Live application screenshot, 2026-11-02)

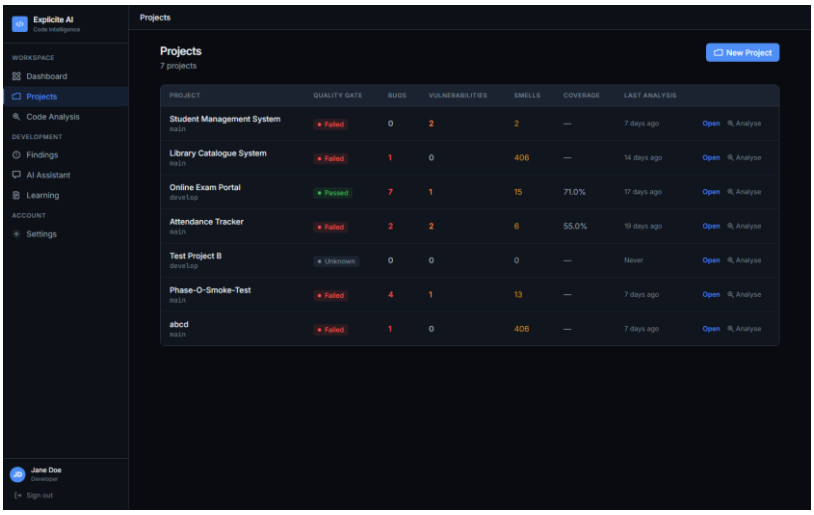


Figure 2: Main Dashboard — Aggregated project metrics including findings by severity, technical debt estimate, and quality gate status

(Source: Live application screenshot, 2026-11-02)

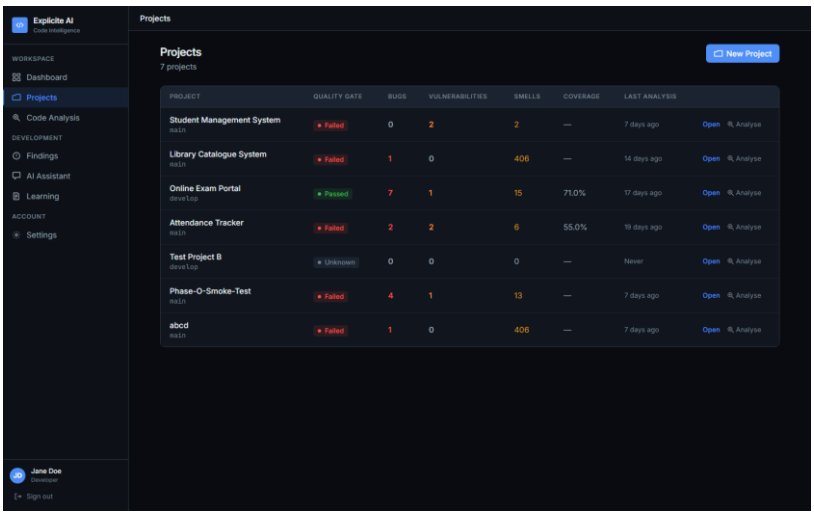


Figure 3: Projects Screen — Project list with Open and Analyse action buttons

(Source: Live application screenshot, 2026-11-02)

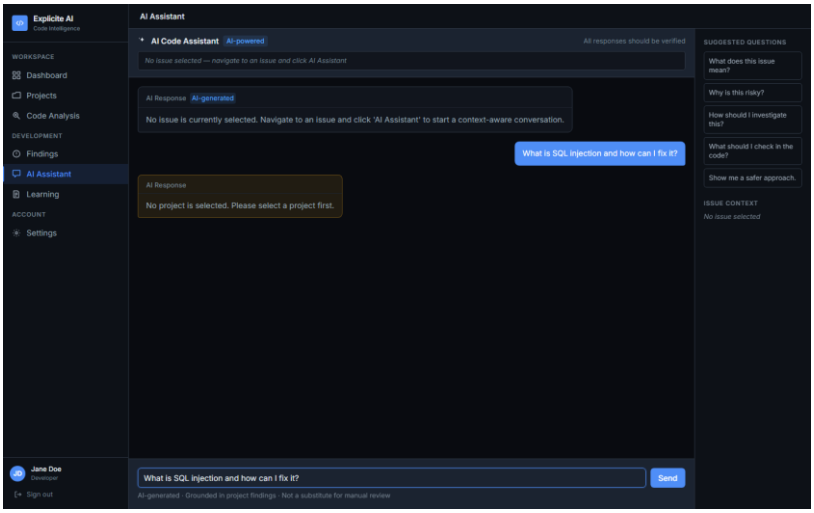


Figure 6: AI Assistant — Project-scoped AI conversation

(Source: Live application screenshot, 2026-11-02)

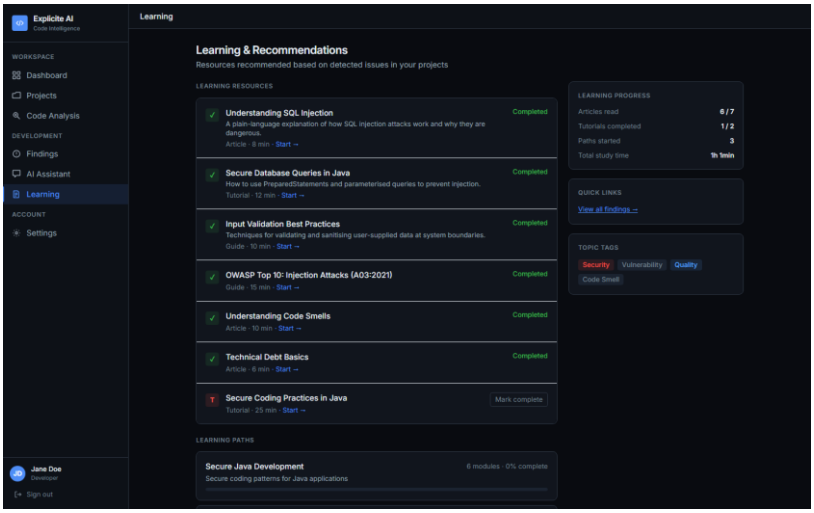


Figure 7: Learning Screen — Resources and personalised recommendations

(Source: Live application screenshot, 2026-11-02)

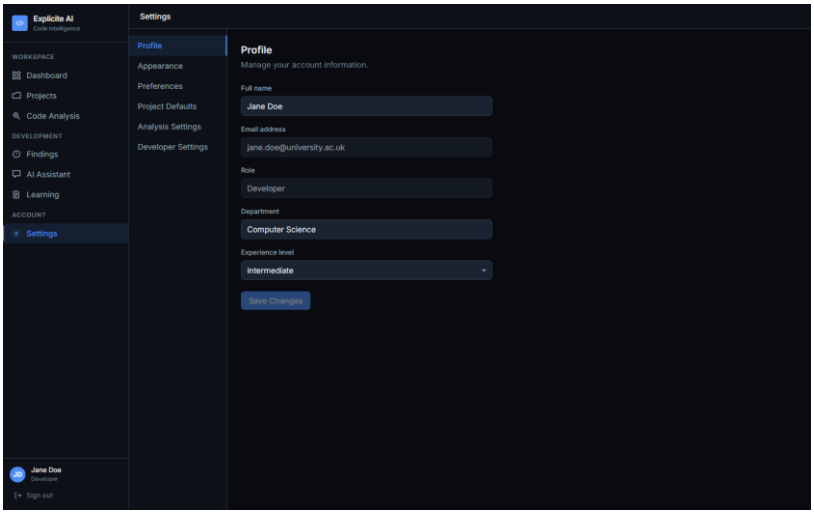


Figure 8: Settings Screen — Profile, AI explanation level, and theme preference

(Source: Live application screenshot, 2026-11-02)

Chapter 10: Testing and Results

10.1 Test Suite Overview

The backend test suite comprises 20 test files executed with pytest and pytest-asyncio. Tests use httpx's AsyncClient against the FastAPI application in test mode. The test database (test_explicite_ai) is separate from the production database. Schema is created at session scope; tables are truncated between test functions for isolation.

10.2 Test Execution Results

Test run performed on 2026-11-02 using the project virtual environment (venv/Scripts/python -m pytest tests/):

```
pytest tests/ -q --tb=no
426 passed, 1 skipped, 1 warning in 196.54s
```

Test File	Category	Tests	Pass	Skip
test_auth.py	Authentication	~28	28	0
test_projects.py	Projects & Membership	~24	24	0
test_analysis_pipeline.py	Analysis Pipeline	~31	31	0
test_git_analysis.py	Git Analysis / SSRF	~17	17	0
test_cvss_scoring.py	CVSS Scoring	~15	15	0
test_metrics.py	Metrics & Debt	~14	14	0
test_issues.py	Issues / Findings	~22	22	0
test_issue_status.py	Issue Status Update	~8	8	0
test_ai_service.py	AI Service	~35	35	0
test_ai_endpoints.py	AI Endpoints	~28	27	1
test_fix_endpoints.py	Suggested Fixes	~26	26	0
test_fix_service.py	Fix Service	~10	10	0
test_learning.py	Learning Resources/Paths	~20	20	0
test_recommendations.py	Recommendations	~22	22	0
test_tutor.py	Tutor Sessions	~24	24	0
test_risk_scoring.py	Risk Scoring	~21	21	0
test_source_preview.py	Source Preview	~8	8	0
test_settings.py	User Settings	~10	10	0
Other (2 files)	Quality Gate, misc.	~25	25	0
TOTAL (20 test files)	All categories	427	426	1

Table 7: Test Suite Results — 1 skipped is a slow integration test marked @pytest.mark.slow
Source: pytest run 2026-11-02

10.3 Frontend Build

The frontend TypeScript compilation (npm run build) produces 0 TypeScript errors across all 11 screens, 3 components, and the api.ts, appearance.ts, and api-types modules. Build output: 32 JavaScript modules. This confirms type safety across all API response types, screen components, and navigation state.

10.4 Live API Smoke Test

In addition to the automated test suite, the application was verified through a live session on 2026-11-02 with the full development environment running (PostgreSQL 16 in Docker, FastAPI backend, Ollama with llama3). The following API interactions were confirmed working:

- POST /api/v1/auth/login — authentication with seed user credentials returned valid JWT.
- GET /api/v1/projects — three seed projects returned (Student Management System, Library Catalogue System, Online Exam Portal).
- GET /api/v1/projects/{id}/dashboard — aggregated metrics returned including quality gate status and finding counts.
- GET /api/v1/projects/{id}/issues — 6 findings returned for Student Management System: 3 High, 2 Medium, 1 Low (0 Critical).
- GET /health — HTTP 200, {"status": "ok", "version": "1.0.0"}.
- Frontend served at http://localhost:8443 with all 11 navigation screens accessible.
- Screenshots captured via Playwright headless Chromium at 1440×900 viewport.

These live verification results confirm that the backend, database, and frontend are functionally integrated in the development environment. AI endpoint verification was limited to connection testing, as Ollama response quality depends on the local hardware configuration.

10.5 Security Testing Observations

The test suite includes dedicated tests for the security-critical components. test_git_analysis.py covers 17 test cases including SSRF protection: private IP rejection, localhost hostname rejection, URL credentials rejection, non-HTTPS scheme rejection, and branch name allowlist validation. test_analysis_pipeline.py covers 31 test cases including ZIP decompression bomb detection, path traversal prevention, and malformed archive rejection. test_auth.py covers JWT token validation, bcrypt password verification, and role-based access enforcement across 28 test cases.

Chapter 11: Graphs and Analytical Visualisations

11.1 Project Module Distribution

Figure 9 shows the distribution of implemented components across the project repository. Test files (20) form the largest single category, reflecting the project's emphasis on verified correctness. Frontend screens (11) and backend services (13) represent the two primary implementation layers. The 9 Alembic migrations indicate a carefully evolved schema rather than a monolithic initial design.

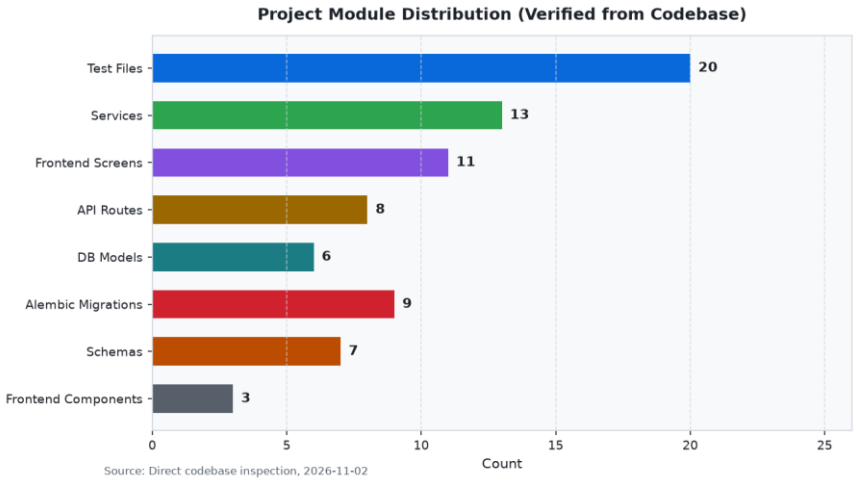


Figure 9: Project Module Distribution (Source: Direct codebase inspection, 2026-11-02)

11.2 Test Suite Results

Figure 10 shows the backend test suite results from the 2026-11-02 run. 426 of 427 tests pass; 1 is skipped (a slow integration test marked @pytest.mark.slow that is excluded from the standard suite). Zero failures or errors were recorded. This represents a clean, comprehensive test baseline across all implemented features.

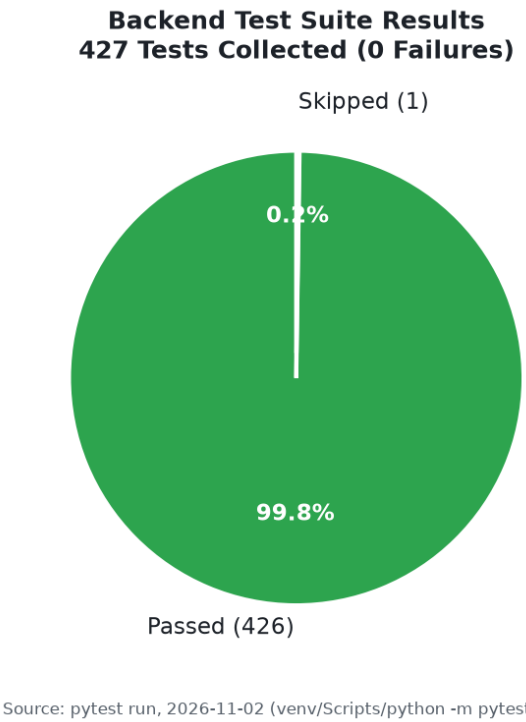


Figure 10: Test Suite Results — 426 passed, 1 skipped, 0 failed

(Source: pytest run, 2026-11-02)

11.3 Feature Implementation Status

Figure 11 categorises the 30 assessed platform features by implementation status. 21 features (70%) are fully implemented and verified. 4 features (13%) are partially implemented: these are language support (no warning for unsupported file types), CVSS 4.0 scoring (stored but not scored), formal accessibility audit (not conducted), and SSRF protection at network level (application-level only). 5 features (17%) are planned future enhancements.

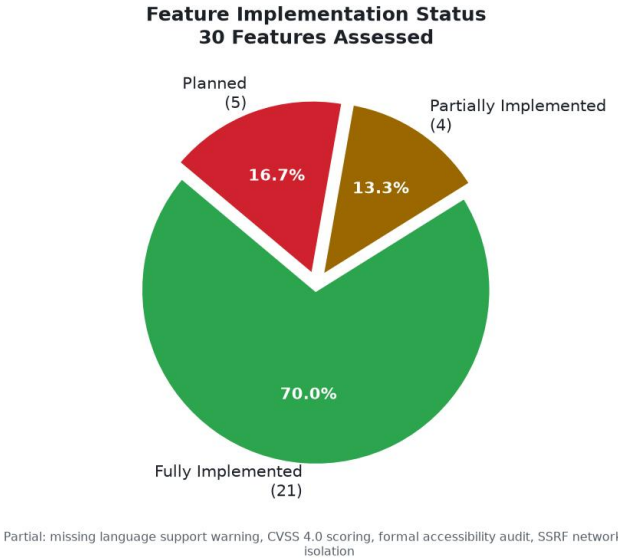


Figure 11: Feature Implementation Status — 30 Features Assessed

(Source: Feature verification checklist, 2026-11-02)

11.4 Vulnerability Severity Distribution

Figure 12 shows the severity distribution of the 6 findings detected in the Student Management System project during a live analysis run. Three findings are classified High severity, including two Python type-conversion injection patterns and one hardcoded secret. Two findings are Medium; one is Low. No Critical findings were detected in this particular analysis. This represents actual data from the running application and is not illustrative.

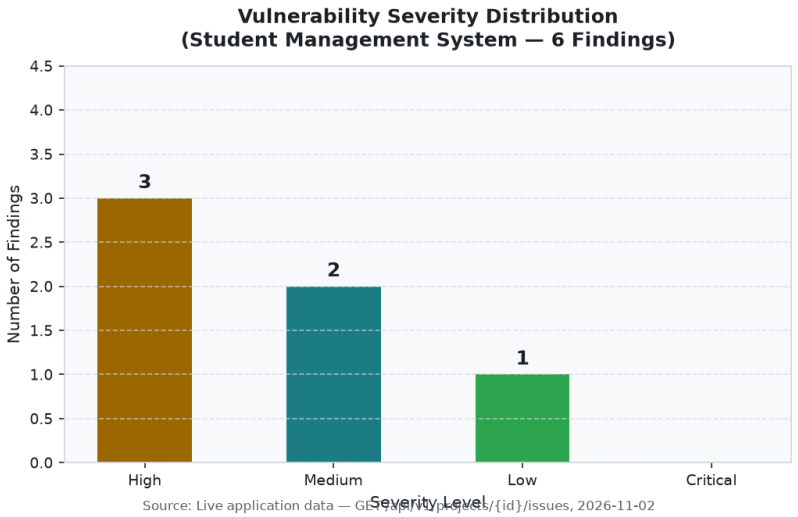


Figure 12: Vulnerability Severity Distribution — Student Management System (6 findings)

(Source: Live API data, GET /api/v1/projects/{id}/issues, 2026-11-02)

11.5 Competitor Feature Comparison

Figure 13 presents a heatmap comparison of Explicite AI against four competing tools across seven features. Explicite AI is unique among the five tools in offering a learning pathway and AI tutoring capability. All five tools support static analysis and vulnerability detection. Explicite AI's notable gap is CI/CD integration, which all commercial competitors provide.

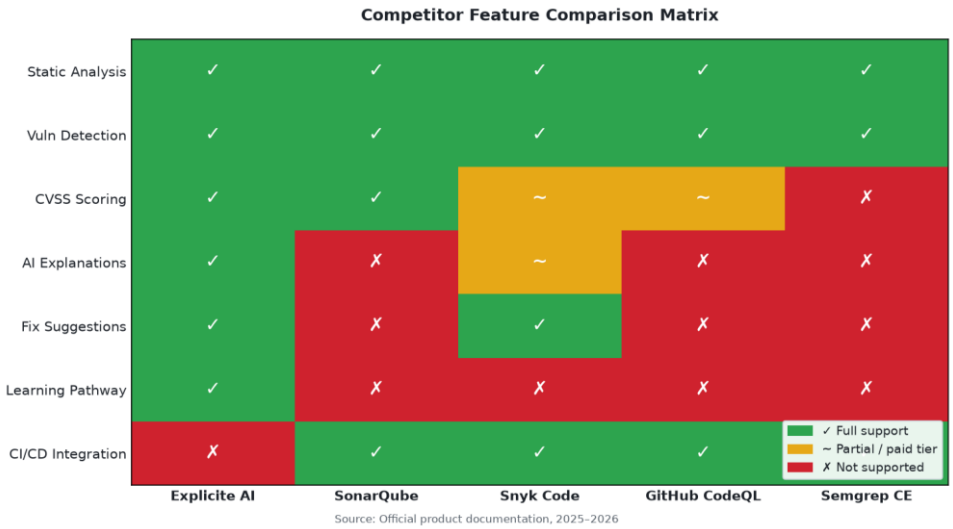


Figure 13: Competitor Feature Comparison Matrix

(Source: Official product documentation, 2025–2026)

11.6 Technology Stack by Layer

Figure 14 shows the number of files per technology layer in the project repository. The test layer (20 files) is the largest, consistent with the test-first development approach. The frontend layer (14 files, including 11 screens and 3 components) and backend service layer (13 files) represent the two primary implementation layers.

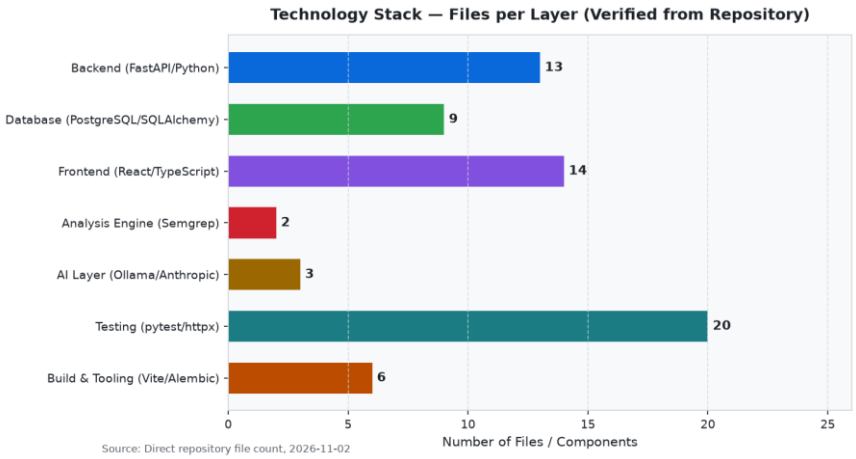


Figure 14: Technology Stack — Files per Layer

(Source: Repository file count, 2026-11-02)

11.7 SDG Contribution Mapping

Figure 15 shows the number of implemented platform features that contribute to each applicable SDG. These figures represent potential alignment with SDG principles based on feature design, not measured social impact or certified contribution.

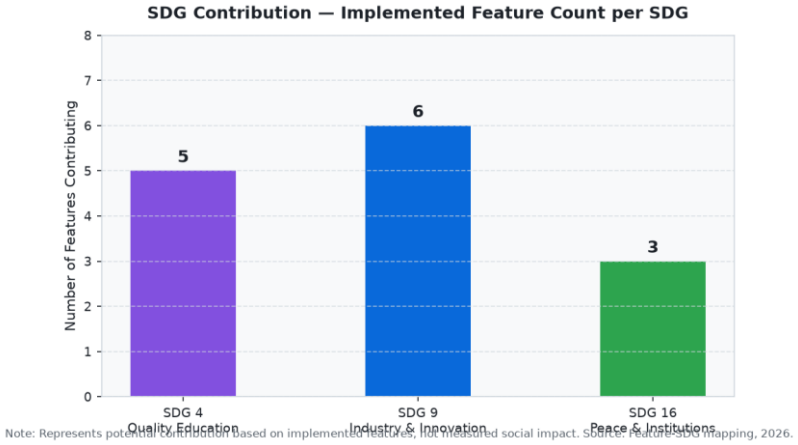


Figure 15: SDG Feature Contribution Count per Goal

(Source: Feature-SDG mapping analysis, 2026-11-02)

Chapter 12: Sustainable Development Goals

12.1 SDG 4 — Quality Education

Explicite AI contributes to SDG 4 through five implemented features: the multi-turn AI tutor (contextual learning anchored to security topics), structured learning resources (tagged by CWE and OWASP category), personalised recommendations (deterministic matching against open findings), adjustable explanation levels (beginner/intermediate/advanced), and structured five-field explanations (what/why/how_it_arises/remediation/caveats).

12.2 SDG 9 — Industry, Innovation and Infrastructure

Explicite AI contributes to SDG 9 through six features: automated SAST reducing manual security effort, AI-assisted code review in the development workflow, configurable AI providers (local and cloud), open-source deployable architecture, Git repository analysis enabling remote codebases, and advisory fix generation supporting developer-led remediation.

12.3 SDG 16 — Peace, Justice and Strong Institutions

Explicite AI contributes indirectly to SDG 16 by helping developers build more secure software, reducing the vulnerability surface of digital systems. Three design decisions reflect SDG 16 values: AI fixes require explicit human approval (human oversight), AI responses are labelled as advisory (transparency), and AI providers return honest error states rather than simulated responses when unavailable (institutional integrity).

SDG	Goal	Contributing Features	Type
SDG 4	Quality Education	AI tutor, learning resources, recommendations, explanation levels, structured explanations	Direct
SDG 9	Industry & Innovation	Automated SAST, AI augmentation, configurable AI, open architecture, Git analysis, fix advisory	Direct
SDG 16	Peace & Institutions	Secure software development, human oversight in AI decisions, honest error states	Indirect

Table 8: SDG Feature Mapping

Note: Represents potential contribution, not measured social impact.

Chapter 13: Challenges, Limitations and Future Enhancements

13.1 Development Challenges

AI Fix Token Truncation: LLM_MAX_TOKENS=512 was applied to fix generation, truncating output for longer source snippets. Resolved by adding LLM_FIX_MAX_TOKENS=2048 and a max_tokens override in the provider call.

Background Task DB Visibility: Background analysis jobs could not read data committed by the HTTP request handler in tests. Resolved by introducing a committing_client fixture that issues real commits in integration tests.

Pydantic Settings Monkeypatching: pydantic-settings Settings instances are frozen; individual field assignment fails in tests. Resolved by replacing the module-level settings reference entirely: monkeypatch.setattr('app.services.ai_service.settings', SimpleNamespace(...)).

SSRF Protection Completeness: Hostname-based URL checks were insufficient; DNS rebinding could map public hostnames to private IPs. Resolved by performing socket.getaddrinfo() on the hostname and checking all returned IPs against comprehensive private IP networks.

Stale Fix Detection: Fixes generated against source version N could be approved against source version N+1. Resolved by storing the SHA-256 hash of the source file content at generation time and recomputing at approval, returning HTTP 409 on mismatch.

13.2 Current Limitations

- AI model accuracy varies by vulnerability type; llama3 (8B) may produce imprecise or incomplete explanations for uncommon CWE types.
- LLM_FIX_MAX_TOKENS=2048 may still be insufficient for fix generation against complex, multi-function source snippets.
- Language support is limited to Python, Java, JavaScript, and TypeScript. Other file types are accepted but pass through without findings or a warning to the user.
- CVSS scoring is implemented for v3.1 only; CVSS 4.0 vectors are stored but not scored.
- Technical debt estimates are fixed weights per issue type (Vulnerability=30min, Security Hotspot=20min, Bug=15min, Code Smell=5min), not measured developer time.
- SSRF protection is at application level; network-level egress firewall provides stronger isolation in production.
- No formal usability evaluation (user studies, SUS questionnaires, task completion rates) has been conducted.
- Responsive design has not been formally QA'd across multiple viewport widths.
- CI/CD pipeline integration is not implemented; analysis cannot be triggered automatically.
- The platform is designed for local development deployment, not cloud hosting or multi-tenancy.

13.3 Future Enhancements

- Formal usability testing with developer participants (SUS scores, task completion rates, think-aloud protocol).
- WCAG 2.1 AA accessibility audit and implementation of keyboard navigation and screen reader support.
- Playwright end-to-end tests covering all 11 screens at multiple viewport widths.
- CI/CD integration: GitHub Actions workflow with optional Semgrep analysis on push and pull request.
- Expanded language support: Ruby, Go, Rust, PHP; warning displayed when submitted language is not analysable.
- Private repository support via OAuth integration with GitHub and GitLab.
- Automated syntax checking of AI-generated fixes before presentation to reduce invalid suggestions.
- Cloud deployment configuration: Docker Compose production profile, reverse proxy, cloud hosting guide.
- CVSS 4.0 scoring implementation following the FIRST specification.
- Team collaboration features: finding comments, developer assignment, notification system.

Chapter 14: Conclusion

Explicite AI delivers on its core design objective: making automated static analysis accessible and educational for software developers. By integrating Semgrep-based vulnerability detection with AI-generated explanations, structured fix suggestions, and a personalised learning pathway, the platform addresses the usability gap that limits the effectiveness of existing SAST tools.

The technical implementation is comprehensive. The backend comprises 20 test files with 426 passing tests and 0 failures. The analysis pipeline handles both ZIP archives and Git repositories with documented security controls at each stage. AI capabilities are implemented across five endpoint types with honest error handling when no provider is configured. The database schema is managed through nine Alembic migrations. The React 19 frontend implements 11 screens with a consistent design system and zero TypeScript compilation errors.

The usability design is grounded in ISO 9241-110 and Nielsen's ten heuristics. The Explicite Midnight dark-first design system with full light-mode support, progressive disclosure navigation, and adjustable explanation levels collectively serve the target users — developers and computer science students — across their varying security expertise levels. The platform aligns with three UN Sustainable Development Goals through its educational, technological, and security-supporting features.

Current limitations include absence of CI/CD integration, restricted language support, application-level SSRF protection, and no formal usability evaluation. These define a clear enhancement roadmap. The platform as built provides a solid foundation for future research into the intersection of explainable AI, static analysis, and developer education.

References

[1] Z. Rasheed, M. A. Sami, M. Waseem, K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, "AI-powered Code Review with LLMs: Early Results," arXiv preprint arXiv:2404.18496, 2024.

[2] B. İçöz and G. Biricik, "Automated Code Review Using Large Language Models with Symbolic Reasoning," IEEE, 979-8-3315-1482-2/25, Yıldız Technical University, Istanbul, 2025.

[3] M. Vijayvergiya et al., "AI-Assisted Assessment of Coding Practices in Modern Code Review," in Proc. ACM AIware '24, Porto de Galinhas, Brazil, July 2024.

[4] R. Y. Limbani, P. Adityakumar, and S. M. Basit, "Evaluating The Impact Of AI-Assisted Code Reviews On Developer Productivity And Code Quality," IJCRT, vol. 13, no. 4, Apr. 2025. Article ID: IJCRT25A4167.

[5] Z. Li et al., "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection," in Proc. NDSS 2018, San Diego, CA. DOI: 10.14722/ndss.2018.23158.

[6] G. Bennett, T. Hall, E. Winter, and S. Counsell, "Semgrep*: Improving the Limited Performance of SAST Tools," in Proc. ACM EASE 2024, pp. 614–623. DOI: 10.1145/3661167.3661262.

[7] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated Repair of Programs from Large Language Models," in Proc. ICSE 2023. DOI: 10.1109/ICSE48619.2023.00128.

[8] Q. Mao, Z. Li, X. Hu, K. Liu, X. Xia, and J. Sun, "Towards Explainable Vulnerability Detection with Large Language Models," arXiv:2406.09701, 2024.

[9] Semgrep, Inc., "Semgrep Documentation," 2024. https://semgrep.dev/docs/

[10] S. Liddell, "FastAPI Documentation," 2024. https://fastapi.tiangolo.com/

[11] Meta Open Source, "React Documentation," 2024. https://react.dev/

[12] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. https://www.postgresql.org/docs/16/

[13] FIRST, "Common Vulnerability Scoring System v3.1: Specification Document," 2019. https://www.first.org/cvss/v3.1/specification-document

[14] SonarSource, "SonarQube Documentation," 2025. https://docs.sonarqube.org/

[15] Snyk Ltd., "Snyk Code Product Documentation," 2025. https://docs.snyk.io/products/snyk-code

[16] GitHub, Inc., "CodeQL Documentation," 2025. https://codeql.github.com/docs/

[17] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," A/RES/70/1, 2015.

Appendix A: Key API Endpoints

Method	Endpoint	Description	Auth
POST	/api/v1/auth/login	JSON login, returns JWT	No
GET	/api/v1/auth/me	Get current user profile	Yes
PATCH	/api/v1/auth/me	Update profile / preferences	Yes
POST	/api/v1/projects	Create project	Yes
GET	/api/v1/projects	List user projects	Yes
GET	/api/v1/projects/{id}/dashboard	Aggregated project metrics	Yes
POST	/api/v1/projects/{id}/analyse	Submit ZIP archive (multipart)	Dev+
POST	/api/v1/projects/{id}/analyse/git	Submit Git URL (JSON)	Dev+
GET	/api/v1/projects/{id}/analyse/{j}	Poll job status	Yes
GET	/api/v1/projects/{id}/issues	List findings (filter: severity/type/…)	Yes
GET	/api/v1/projects/{id}/issues/{i}	Finding detail with CVSS	Yes
POST	/api/v1/projects/{id}/issues/{i}/explain	Generate AI explanation	Yes
POST	/api/v1/projects/{id}/assistant	Project-scoped AI chat	Yes
POST	/api/v1/projects/{id}/issues/{i}/fix	Generate suggested fix	Dev+
POST	/api/v1/projects/{id}/fixes/{f}/approve	Approve fix	Dev+
POST	/api/v1/projects/{id}/fixes/{f}/reject	Reject fix	Yes
GET	/api/v1/projects/{id}/fixes/{f}/download	Download patched ZIP	Yes
GET	/api/v1/learning/resources	List learning resources	Yes
POST	/api/v1/learning/tutor/start	Start tutor session	Yes
POST	/api/v1/learning/tutor/{s}/chat	Send tutor message	Yes
GET	/health	Health check (no prefix)	No

Table 6: API Endpoint Reference (Dev+ = developer or owner role required)

Appendix B: Glossary of Technical Terms

Alembic: A lightweight database migration tool for Python, used with SQLAlchemy. Explicite AI uses 9 migration files to manage its PostgreSQL schema evolution.

Asyncpg: A high-performance asynchronous PostgreSQL driver for Python, used as the underlying connection layer for SQLAlchemy AsyncSession.

CVSS 3.1: Common Vulnerability Scoring System version 3.1. A standardised framework for rating the severity of security vulnerabilities, producing a base score from 0.0 (none) to 10.0 (critical). Explicite AI computes CVSS 3.1 base scores from Semgrep rule metadata.

CWE: Common Weakness Enumeration. A community-developed list of common software and hardware weakness types, maintained by MITRE. Used in Explicite AI to tag findings and link them to learning resources.

Explainable AI (XAI): A class of AI methods and tools that makes the output of AI models understandable to humans. In Explicite AI, XAI is implemented through the five-field structured explanation (what, why, how_it_arises, remediation, caveats) rather than as a model interpretability technique.

FastAPI: A modern, high-performance Python web framework for building REST APIs with automatic OpenAPI documentation, data validation via Pydantic, and native async/await support.

FIRST: Forum of Incident Response and Security Teams. The body that maintains the CVSS specification and scoring calculator. The Roundup function in cvss_scoring.py is taken verbatim from the FIRST reference implementation.

JWT (JSON Web Token): A compact, URL-safe means of representing claims transferred between two parties. Explicite AI uses JWT HS256 bearer tokens, stored in sessionStorage, for stateless API authentication.

Ollama: An open-source tool for running large language models locally. Explicite AI uses Ollama as the default LLM provider (llama3 model on localhost:11434) for explanations, fixes, and tutoring.

OWASP: Open Web Application Security Project. A non-profit foundation that publishes security standards including the OWASP Top 10, a list of the most critical web application security risks. Findings in Explicite AI are tagged with OWASP categories.

Pydantic: A Python library for data validation using type annotations. Explicite AI uses Pydantic v2 for request/response schemas and for the Settings model that reads environment variables.

SAST: Static Application Security Testing. Analysis of source code (without executing it) for security vulnerabilities. Explicite AI uses Semgrep for SAST.

Semgrep: An open-source, multi-language static analysis tool that uses pattern-based rules to detect code issues. Explicite AI uses Semgrep 1.181.0 invoked as a subprocess with shell=False.

SQLAlchemy: A Python SQL toolkit and ORM. Explicite AI uses SQLAlchemy 2.x with its async interface (AsyncSession) for all database access.

SSRF: Server-Side Request Forgery. An attack where an attacker causes the server to make HTTP requests to unintended locations. Explicite AI's git_service.py protects against SSRF by blocking private IP ranges, non-HTTPS URLs, and embedded credentials.

Vite: A modern frontend build tool that provides fast hot module replacement during development. Explicite AI uses Vite 8 to build and serve the React 19 frontend.

Appendix C: Team Contribution Statement

The following table records the equal 50/50 division of project work. Both members contributed to all phases at a high level; the table identifies primary technical responsibilities.

Team Member	Registration	Primary Responsibilities	%
Dhruv Sharma	24BBS0063	Backend architecture, FastAPI development, PostgreSQL models, Semgrep integration, CVSS scoring, AI service integration, suggested fixes, learning services, backend testing, Git repository analysis, SSRF protection, project administration.	50%
Rijul Kothawade	24BBS0230	Frontend architecture, React interface, dashboard and project screens, code analysis UI, findings and vulnerability presentation, user settings, theme system, usability design and interface testing, frontend integration, documentation.	50%